ALLIANCE UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

DATABASE MANAGEMENT SYSTEM

ASSIGNMENT 5

DATABASE CREATION AND TABLE IMPLEMENTATION

Hotel Management System

Particular Details	**Information**
Student Name	A Praisy Emberline
Programme	BCA – Artificial Intelligence & Machine Learning
Subject	Database Management System
University	Alliance University
Academic Year	2nd Year

Introduction

Database creation and table implementation are important steps in developing a database management system. SQL Data Definition Language (DDL) commands create databases, define tables, select data types, and establish relationships. This assignment implements the eight tables from the Hotel Management System reference using MySQL.

1. Creating the Database

```
CREATE DATABASE Hotel_Management_DB;
USE Hotel_Management_DB;
```

CREATE DATABASE creates a database, while USE selects it for the following commands.

2. Creating the Required Tables

Table 1: Guest

```
CREATE TABLE Guest (
  guest_id INT PRIMARY KEY,
  guest_name VARCHAR(100) NOT NULL,
  email VARCHAR(100),
  phone VARCHAR(15),
  address VARCHAR(200)
);
```

guest_id uniquely identifies each guest. The remaining columns store contact and address information.

Table 2: Room_Type

```
CREATE TABLE Room_Type (
  room_type_id INT PRIMARY KEY,
  type_name VARCHAR(50) NOT NULL,
  price_per_night DECIMAL(10,2) NOT NULL,
  description VARCHAR(200)
);
```

Room_Type stores the category and standard nightly price for each type of room.

Table 3: Room

```
CREATE TABLE Room (
  room_id INT PRIMARY KEY,
  room_number VARCHAR(10) NOT NULL,
  floor_number INT,
  room_status VARCHAR(30) NOT NULL,
  room_type_id INT NOT NULL,
  FOREIGN KEY (room_type_id)
    REFERENCES Room_Type(room_type_id)
);
```

Each room belongs to a room type. room_type_id is a foreign key referencing Room_Type.

Table 4: Staff

```
CREATE TABLE Staff (
  staff_id INT PRIMARY KEY,
  staff_name VARCHAR(100) NOT NULL,
  role VARCHAR(50),
  phone VARCHAR(15),
  department VARCHAR(50)
);
```

Staff stores employee details. In the supplied reference, Staff is an independent operational entity and has no defined foreign-key relationship to Reservation.

Table 5: Reservation

```
CREATE TABLE Reservation (
  reservation_id INT PRIMARY KEY,
  guest_id INT NOT NULL,
  room_id INT NOT NULL,
  check_in_date DATE NOT NULL,
  check_out_date DATE NOT NULL,
  reservation_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (guest_id) REFERENCES Guest(guest_id),
  FOREIGN KEY (room_id) REFERENCES Room(room_id)
);
```

Reservation stores booking dates and status. The foreign keys connect each reservation to a guest and a room.

Table 6: Service

```
CREATE TABLE Service (
  service_id INT PRIMARY KEY,
  service_name VARCHAR(100) NOT NULL,
  service_price DECIMAL(10,2) NOT NULL,
  description VARCHAR(200)
);
```

Service stores optional services offered by the hotel and their prices.

Table 7: Payment

```
CREATE TABLE Payment (
  payment_id INT PRIMARY KEY,
  reservation_id INT NOT NULL,
  payment_date DATE NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(30),
  payment_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (reservation_id)
    REFERENCES Reservation(reservation_id)
);
```

Payment records payment details for a reservation. Multiple payment records per reservation are allowed in this schema, matching the supplied reference.

Table 8: Service_Usage

```
CREATE TABLE Service_Usage (
  usage_id INT PRIMARY KEY,
  reservation_id INT NOT NULL,
  service_id INT NOT NULL,
  usage_date DATE NOT NULL,
  quantity INT NOT NULL,
  FOREIGN KEY (reservation_id)
    REFERENCES Reservation(reservation_id),
  FOREIGN KEY (service_id)
    REFERENCES Service(service_id)
);
```

Service_Usage records services consumed during a reservation. It connects Reservation and Service.

3. Defining Relationships Between Tables

Relationship	Cardinality	Purpose
Room_Type → Room	One-to-Many	One room type can be assigned to several rooms.
Guest → Reservation	One-to-Many	One guest can have multiple reservations over time.
Room → Reservation	One-to-Many over time	A room can appear in multiple reservations on different dates.
Reservation → Payment	One-to-Many	A reservation may have multiple payment records in this design.
Reservation → Service_Usage	One-to-Many	A reservation can have several service-usage records.
Service → Service_Usage	One-to-Many	A service can appear in many usage records.

4. Verifying Table Creation

```
SHOW TABLES;
DESCRIBE Guest;
DESCRIBE Room_Type;
DESCRIBE Room;
DESCRIBE Staff;
DESCRIBE Reservation;
DESCRIBE Service;
DESCRIBE Payment;
DESCRIBE Service_Usage;
```

SHOW TABLES lists tables in the selected database. DESCRIBE displays the columns, data types, and key information for a table.

Conclusion

The Hotel_Management_DB database is implemented using eight tables. Primary keys identify records, and foreign keys connect rooms to room types, reservations to guests and rooms, payments to reservations, and service usage to reservations and services. The resulting structure provides a foundation for inserting data and running SQL queries.